

Oct 05, 22 18:42

main.py

Page 1/4

#!/bin/env python3.8

"""

Michael Bentivegna "The Ponderer"
 Deep Learning Assignment 4
 Professor Curro

Initially, I used a GoogLeNet architecture for my model, but the runtime on my computer would have taken approximately 24 hours to complete (so I ended up scrapping it). Then, I implemented simpler inception model with dimension reduction in hopes to make the time complexity more manageable. Although the runtime was reduced to a few hours, it suffered from significant overfitting even with the regularization and normalization techniques I put into place. Finally, I went back to a sequential model architecture (similar to the MNIST dataset), but added data augmentation in hopes to increase accuracy and reduce overfitting on the training set. With this method, I was able to achieve about 87% accuracy on CIFAR10 and 83% for CIFAR100.

"""

```
import os
import logging
```

```
import keras
from tensorflow.keras.layers import BatchNormalization
from keras.layers.core import Dense, Flatten, Dropout
from keras.layers.convolutional import Conv2D, MaxPooling2D
from keras.models import Sequential
import tensorflow as tf
```

```
import numpy as np
from absl import app
from absl import flags
import pickle
```

```
script_path = os.path.dirname(os.path.realpath(__file__))
```

```
# Define default values for flags
FLAGS = flags.FLAGS
flags.DEFINE_integer("batch_size", 32, "Number of samples in batch")
flags.DEFINE_integer("num_epochs", 30, "Number of SGD iterations")
flags.DEFINE_bool("debug", False, "Set logging level to debug")
```

```
def process_data(data):
    """
```

```
    Helper function for getData() to put pixel information in proper form
    """
```

```
        x = np.transpose(
            data.reshape([-1, 32, 32, 3], order="F").astype("float32") / 255, (0, 2, 1, 3)
        )
```

```
        return x
```

```
def getData(folder_name):
    """
```

```
    Get pixel and label data for c10 and c100
    """
```

```
        train_data = np.array([]).reshape(0, 32, 32, 3)
        train_labels = []
```

```
        test_data = np.array([]).reshape(0, 32, 32, 3)
        test_labels = []
```

Oct 05, 22 18:42

main.py

Page 2/4

```
file_list = os.listdir(script_path + "/" + folder_name)
```

```
for file in file_list:
```

```
    path_helper = script_path + "/" + folder_name + "/" + file
```

```
    if file.__contains__("batch_") or file.__contains__("train"):
```

```
        with open(path_helper, "rb") as fo:
```

```
            test = pickle.load(fo, encoding="bytes")
```

```
            # print(np.array(test[b"data"])[1, 1])
```

```
            # print(np.array(test[b"data"])[1, 1025])
```

```
            # print(np.array(test[b"data"])[1, 2049])
```

```
            # print(process_data(np.array(test[b"data"])[1, 1, 0, :]))
```

```
        train_data = np.concatenate(
```

```
            (train_data, process_data(np.array(test[b"data"]))), axis=0
```

```
        )
```

```
        if file.__contains__("batch_"):
```

```
            train_labels = np.concatenate(
```

```
                (train_labels, test[b"labels"]), axis=0
```

```
            )
```

```
        elif file.__contains__("train"):
```

```
            train_labels = np.concatenate(
```

```
                (train_labels, test[b"fine_labels"]), axis=0
```

```
            )
```

```
    elif file.__contains__("test"):
```

```
        with open(path_helper, "rb") as fo:
```

```
            test = pickle.load(fo, encoding="bytes")
```

```
        test_data = np.concatenate(
```

```
            (test_data, process_data(np.array(test[b"data"]))), axis=0
```

```
        )
```

```
        if file.__contains__("test_batch"):
```

```
            test_labels = np.concatenate((test_labels, test[b"labels"]), axis=0)
```

```
        else:
```

```
            test_labels = np.concatenate(
```

```
                (test_labels, test[b"fine_labels"]), axis=0
```

```
            )
```

```
    return (
```

```
        train_data[10000:, :, :, :],
```

```
        train_labels[10000:],
```

```
        train_data[:10000, :, :, :],
```

```
        train_labels[:10000],
```

```
        test_data,
```

```
        test_labels,
```

```
    )
```

```
def conv_layer_helper(filters, model, drop, pool):
```

```
    """
```

```
    Helper Function to build CNN with regularization / normalization techniques
    """
```

```
    model.add(
```

```
        Conv2D(
```

```
            filters,
```

```
            kernel_size=(3, 3),
```

```
            padding="same",
```

```
            kernel_regularizer=keras.regularizers.l2(0.001),
```

```
            activation="relu",
```

```
        )
```

Oct 05, 22 18:42

main.py

Page 3/4

```

    )
    model.add(BatchNormalization())
    model.add(
        Conv2D(
            filters,
            kernel_size=(3, 3),
            padding="same",
            kernel_regularizer=keras.regularizers.l2(0.001),
            activation="relu",
        )
    )
    model.add(BatchNormalization())
    model.add(MaxPooling2D(pool_size=(pool, pool), padding="same"))
    model.add(Dropout(drop))

    return model

def define_cX_model(num_labels):
    """
    Create sequential model for cifar10 or cifar100 datasets
    """
    model = Sequential()

    # Helper function calls to add onto model
    conv_layer_helper(32, model, 0.1, 2)
    conv_layer_helper(64, model, 0.1, 2)
    conv_layer_helper(128, model, 0.1, 2)

    # Dense layers for classification
    model.add(Flatten())
    model.add(Dropout(0.2))
    model.add(
        Dense(1024, activation="relu", kernel_regularizer=keras.regularizers.l2(0
))
    )
    model.add(Dropout(0.2))

    model.add(Dense(num_labels, activation="softmax"))

    return model

def run_analysis(name, size, metrics):
    """
    Avoids DRY code for cifar10 and cifar100 when processing the data and fitting the model
    """

    # Get data
    (
        train_d_cX,
        train_l_cX,
        val_d_cX,
        val_l_cX,
        test_d_cX,
        test_l_cX,
    ) = getData(name)

    # Run model on non-augmented data
    model = define_cX_model(size)

    model.compile(

```

Oct 05, 22 18:42

main.py

Page 4/4

```

        optimizer="adam", loss="sparse_categorical_crossentropy", metrics=metrics
    )
    model.fit(
        train_d_cX,
        train_l_cX,
        validation_data=(val_d_cX, val_l_cX),
        epochs=FLAGS.num_epochs,
        batch_size=FLAGS.batch_size,
        verbose=1,
    )

    # Re-run model with augmented data
    data_generator = tf.keras.preprocessing.image.ImageDataGenerator(
        width_shift_range=0.1, height_shift_range=0.1, horizontal_flip=True
    )

    train_generator = data_generator.flow(train_d_cX, train_l_cX, FLAGS.batch_si
ze)

    model.fit(
        train_generator,
        validation_data=(val_d_cX, val_l_cX),
        batch_size=FLAGS.batch_size,
        epochs=FLAGS.num_epochs,
    )

    print("-----Test Set for CIFAR--" + str(size) + "-----")
    model.evaluate(test_d_cX, test_l_cX, verbose=1)

def main(a):
    """
    Calls run_analysis function for both datasets
    """
    logging.basicConfig()

    if FLAGS.debug:
        logging.getLogger().setLevel(logging.DEBUG)

    # Carries out necessary functions for each dataset
    run_analysis("c10", 10, ["accuracy"])
    run_analysis("c100", 100, ["accuracy", "sparse_top_k_categorical_accuracy"])

if __name__ == "__main__":
    app.run(main)

```